Fitra
Release Plan

Version: <1.1>
Last Modified: <04/11/2026>

APPROVALS

Submitting Organization's Approving Authority:

Signature: GQ Printed Name: Quah YiKai Gary
Date: 21/10/2026 Phone Number: +65 xxxx xxxx
Project Manager

CMS' Approving Authority:

Signature: MW Printed Name: Matthew Wong
Date: 21/10/2026 Phone Number: +65 xxxx xxxx
Lead developer

REVISION HISTORY

Version	**Date**	**Approved by**	**Description of Changes**
0.1	15/10/2026	Gary	Initial template
0.2	21/10/2026	Gary	Modifications to template
0.3	25/10/2026	Gary	Modifications to template
1.0	28/10/2026	Gary	Baseline Version
1.1	04/11/2026	Gary	Revised 5.2 release content to distinguish the 1.0.0 functional milestone, 1.0.1 Lab 5 fixes/deployment/documentation and 2.0.0 future enhancements

			outside the current commitment.

TABLE OF CONTENTS

APPROVALS 2
REVISION HISTORY 2
TABLE OF CONTENTS 3
LIST OF FIGURES 4
LIST OF TABLES 4
1. INTRODUCTION 4
3. OVERVIEW 5
- **3.1 System Purpose and Context** 5
- **3.1 Development History** 5
- **3.3. System Architecture Overview** 6
- **3.4. High-Level System Context** 6
- **3.5. Release Context** 7

4. ASSUMPTIONS, CONSTRAINTS, RISKS 8
- **4.1 Assumptions** 8
- **4.2 Constraints** 8
- **4.3. Risks** 8

5. RELEASE APPROACH 8
- **5.1 Rationale** 8
- **5.2. Release Strategy** 9
  - **5.2.1. Release Content** 9
  - **5.2.2. Release Schedule** 10
  - **5.2.3. Release Impacts** 11
  - **5.2.4. Release Notification** 12

6. GLOSSARY 12

LIST OF FIGURES

Figure 1: System Context Diagram 7

LIST OF TABLES

Table 1: Referenced Documents 1

1. INTRODUCTION

Fitra is a health tracking application developed as part of the SC3040 Advanced Software Engineering project. This Release Plan defines the activities, responsibilities and procedures required to prepare, verify and deliver a release of the application.

The scope of this document covers the planning and coordination of release activities, including release preparation, testing, deployment and post-release maintenance. It establishes a common reference for the project team to ensure that releases are managed systematically and that the application meets the expected quality requirements before delivery.

The intended audience includes project team members responsible for software development, quality assurance, release management and project coordination, as well as the course instructor and evaluators reviewing the project's software engineering deliverables.

Security and privacy considerations are important throughout the release process. The team must ensure that sensitive credentials and configuration values are handled securely, access controls are appropriately configured, and user data is protected against unauthorised access. Testing and release activities should use suitable test accounts and avoid exposing real personal or health-related information.

This document will be maintained throughout the project to reflect changes in release requirements, project scope, testing procedures and deployment arrangements. Updates will be made as necessary to ensure that the Release Plan remains consistent with the application's development progress and the team's release decisions.

3. OVERVIEW

3.1 System Purpose and Context

Fitra is a health tracking application developed as part of the SC3040 Advanced Software Engineering project. Its purpose is to provide users with a platform to monitor their health-related activities while supporting social interaction through friend management features.

The application is designed to provide an accessible user interface for users to interact with the system and manage their information.

The system comprises a frontend, backend services and a database and authentication service. These components work together to support user interactions, process application requests and store relevant information. The release of Fitra requires these components to be configured and verified to ensure that the application functions as intended.

3.1 Development History

Fitra was developed incrementally by the project team, with individual features implemented and integrated throughout the software development process. Development activities included implementing user authentication, registration and navigation, alongside social functionality such as sending and managing friend requests and removing friends.

The application was developed using a collaborative workflow with Git and GitHub for version control and source code management. Separate development branches allow team members to work on individual features before integrating their changes into the main codebase. Testing and debugging are performed during development to identify defects and verify that implemented functionality works as expected.

As development progresses, the team integrates new features and resolves issues discovered during testing. The Release Plan provides a structured approach to preparing the application for delivery and verifying that the integrated system meets the project's requirements.

3.3. System Architecture Overview

Fitra uses Next.js and Tailwind CSS for the frontend, FastAPI for backend services, and Supabase for authentication and database management. Each component has a distinct responsibility within the system.

The frontend provides the user interface through which users interact with Fitra. The backend processes application requests and implements the relevant application logic. Supabase provides

authentication and database services to support user identity management and data persistence. These components communicate to provide the application's functionality.

This separation of responsibilities allows individual components to be maintained and enhanced with reduced impact on other parts of the system. Before release, the integration between these components must be verified to ensure that requests are processed correctly and that data access is appropriately controlled.

3.4. High-Level System Context

The system context diagram illustrates the relationship between Fitra, its users and the external services on which it depends. Users interact with Fitra through the frontend, while the application communicates with the backend and Supabase services to process requests, authenticate users and manage persistent data.

The context diagram provides a high-level representation of the system boundary and its external interactions. It serves as a reference for understanding the components involved in the release and identifying the interfaces that require verification during testing.

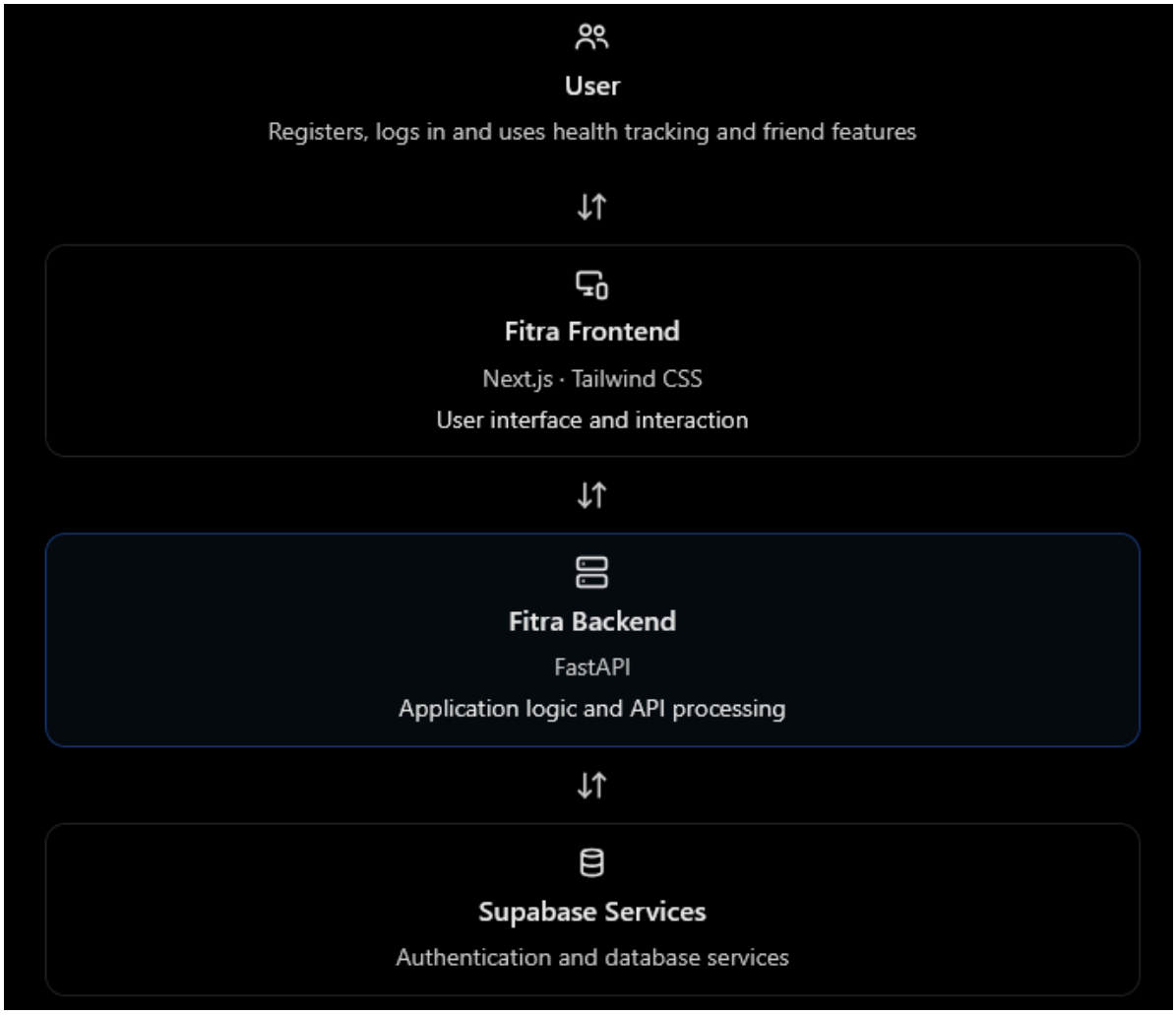


Figure 1. System Context Diagram

3.5. Release Context

The release process covers the integration and verification of the Fitra application and its supporting services. Before delivery, the team should verify the application's core functionality, authentication flow, database access permissions and interactions between the frontend and backend.

The release must also account for environment-specific configuration, including service endpoints and credentials. Any known defects, incomplete functionality or deployment limitations should be documented so that the team and evaluators understand the state of the application at the time of release.

4. ASSUMPTIONS, CONSTRAINTS, RISKS

The releases are bound by fixed lab dates, free-tier services and one shared database; the main risks are outages and conflicting changes near a deadline.

4.1 Assumptions

- All six team members stay available until the Lab 5 presentation on 29 October 2026.
- Supabase, Vercel, Render, Google Gemini and Strava keep their free tiers at current limits.
- Fitra has no public users yet: releases are used by the team, the lab tutor and test accounts.
- Every feature shares one Supabase project, so a database change for one feature can affect the others.

4.2 Constraints

- **Schedule:** Release 1.0 is due at the Lab 4 demo (15 October), Release 1.1 before Lab 5 (29 October), and the project ends on 1 November 2026.
- **Budget:** $0. The project plan uses only free tiers.
- **One database:** there is no separate test database, so each migration runs once on the shared database and must be safe to run twice.
- **Hosting limits:** Render's free tier stops the backend when idle, so the first request after a pause is slow; Gemini and Supabase limit request rates.

4.3. Risks

Risk	Description	Mitigation strategy
Gemini unavailability	The Gemini API might not be readily available due to excessive use by other subscribers (Error 503).	The application will perform retries with other models and fallback to mock data for base responses, notifying the user that the response is not optimal.

5. RELEASE APPROACH

Fitra is released by function: each release adds or hardens a set of features for every user, and is tagged in Git and deployed from main.

5.1 Rationale

- **Fixed milestones:** the Lab 4 demo (15 October) and the Lab 5 presentation (29 October) are the release points in the Project Plan.
- **Agile sprints:** six members built features in parallel and merged each when ready, so a release gathers a set of finished features.
- **Lessons from Release 1.0:** 35 pull requests in 14 days caused merge conflicts and two database table clashes. So Release 1.1 adds no features, and every release starts with a two-day code freeze.
- **Risks in 4.3:** outages and late conflicts favour small, tested releases over one big release at the end.

5.2. Release Strategy

The release strategy for Fitra is a phased function rollout. Fitra's releases are made available in full to the whole audience of the system at once for the team members and test accounts used for demonstrations. There is therefore no pilot group, no staged user migration and no partial availability of a release.

Delivery is segmented by function. Each release is a set of features that have been completed, reviewed and merged into the main branch by the time the code freeze begins. A feature that is not finished by the freeze is held back and carried to the next release rather than delaying the release date, because the release dates are fixed by the laboratory schedule and cannot move.

Each release is cut from the main branch, recorded as an annotated Git tag and a GitHub release, and deployed from that tag to Vercel and Render. The content of each release is listed in its Version Delivery Description, as described in the Software Configuration Management Plan [R3].

5.2.1. Release Content

The project has progressed through multiple releases, with each release building on the previous version. Releases 0.1.0 and 1.0.0 represent earlier development milestones, while release 1.0.1 covers the fixes, deployment improvements and testing activities for Lab 5. Release 2.0.0 represents potential future enhancements outside the current project commitment. The requirement identifiers below refer to the functional requirements in the System Requirement Specification [R2].

Release 1.0.0, delivered at the Lab 4 demonstration on 15 October 2026. This release delivers the full functional scope of the product. It includes user registration, login and logout (2.1.1); the friend list, friend search and friend removal (2.1.2); the home dashboard with health data overview and manual modification, the health and food consumption overviews and the daily check-in (2.1.3); the settings page with the BMR, TDEE and macronutrient calculation and Strava account linking (2.1.4); meal analysis from a photograph with manual review of the

estimate (2.1.5); the conversational assistant (2.1.6); the training routine page with readiness scoring, plan generation, the daily recommended workout, session logging, the schedule view, rescheduling and plan recalculation (2.1.7); and the gamification page with experience points, achievements and the leaderboard (2.1.8).

Release 1.0.0 does not include live synchronisation from a wearable device, support for health export formats other than those stated in the System Requirement Specification.

Release 1.0.1, delivered before the Lab 5 presentation on 29 October 2026. This release adds no new functionality and includes defect fixes across existing features, improvements to input validation, resolution of issues identified with the database tables during Release 1.0.0, and completion of the project documentation set.

5.2.2. Release Schedule

All dates use Singapore time. The following is a proposed working schedule for the remaining milestones; i.e confirm exact cut-off times and member availability. A failed checkpoint means no release authorisation, even if the milestone date is close.

Table 4: Release Schedule

Date (SGT)	**Activity/gate**	**Responsible**	**Required output**
21–23 Oct 2026	Confirm baseline, hosting/access, migration state and accepted 1.0.1 defects.	Gary; Matthew, Quah Yew Yi	Approved scope and accountable Jira issues; baseline/tag identified.
24–26 Oct 2026	Complete accepted fixes; integrate feature paths; rehearse candidates and prepare recovery.	Feature owners, QA engineers	Reviewed PRs, candidate commit, migration plan and test results.
27 Oct 2026	Begin two-day freeze; exact cut-off to be confirmed.	Quah Yew Yi, Gary	Candidate locked; CCB approval required for release-blocking exceptions.
27–28 Oct 2026	Run release checkpoints, live smoke/ownership checks and go/no-go	Matthew, Lau Jing Jie, Gary, Quah Yew Yi	Technical/QA decisions, delivery authorisation, release

	review; deploy only after authorisation.		record and verified environment.
29 Oct 2026	Recheck services and conduct Lab 5 presentation.	All members; Gary coordinates	Demonstrate verified tag with disclosed limitations; retain evidence.
30 Oct–1 Nov 2026	Archive final delivery and complete handover.	Quah Yew Yi; Gary; all owners	Controlled source/documents, results, known issues and access handover.

5.2.3. Release Impacts

Release	Systems and data affected	Effects on users	Process change	Goal
0.1.0	New Vercel, Render and Supabase projects	None: no users yet	Work moves to feature branches merged into main by pull request	A working skeleton every feature can build on
1.0.0	All eight features go live together. 15 migrations change the shared database (training 8, gamification 5, meal analysis 2); keys for Supabase, Strava and Gemini are set on Vercel and Render; the backend accepts calls only from the web app's address	Test accounts see every feature for the first time. Existing data is kept, and past workouts, meals and check-ins earn XP once	No change from 0.1.0	All SRS use cases are working for the Lab 4 demo
1.0.1	Fixes across existing features; a working Render deployment; a CI workflow and branch protection on GitHub. No major database changes are	None expected: fixes only	From the post-Lab 4 baseline, every change needs an approved CR, a passing CI run and a reviewed	Deliver the Lab 5 release with a working deployment and all required Lab 5 test cases passing.

	planned.		pull request; main is frozen from 27 to 29 October	
2.0.0	New tables for health imports and new gamification features; Google Fit and Apple Health export files as a new input; new SRS requirements	New features; existing data is kept	New requirements go into the SRS and through the CCB before work starts	More ways to bring in data, and more reasons to return each day

5.2.4. Release Notification

Group affected	**Notification window**	**Description**	**Method of notification**
Users	7 days prior to release of update	Users using the application will be affected by the release of a new version and will be notified in advance.	Email and in-app notification
Stakeholders	14 days prior to release of update	Review and approve deployment readiness and sign-off on planned operational impacts	Email and physical meeting.

6. GLOSSARY

Term	**Definition**
API (Application Programming Interface)	The API acts as a middleman between the client and server, allowing for communication of information.
Continuous Integration (CI)	The process in which developers merge code changes into a shared repository.
Change Request	A proposal to alter the project scope, features or timelines.

(CR)	
Pull Request (PR)	A request to review and merge code changes from one branch into another
Vercel	A frontend deployment platform to manage the user interface
Render	A backend deployment platform to manage the server
GitHub	A version control application used in managing the codebase.
FastAPI	A Python web framework used for building Fitra's RESTful services
Next.js	A React framework used to build Fitra's frontend
Tailwind CSS	Styling framework used to design a responsive user interface within the frontend
Gemini	Google's AI service used by FITRA for supported AI features, including meal-image analysis
Supabase	The service used by FITRA for authentication and PostgreSQL database storage.